

POS System Database Architecture

Designed from the current mock frontend requirements

Overview

The mock frontend shows a retail POS system with product catalog, customer management, supplier management, inventory control, sales/returns, invoices, users, and store register operations. The database should therefore be relational and normalized, with transactional tables for sales and master data for all entities.

Recommended database stack

PostgreSQL via Supabase is the best fit for this project because the stack already targets Supabase and the application requires transactional integrity, reporting, and relational joins across products, orders, inventory, and customers.

Layer	Purpose
Stores	Store locations and branch metadata
Users & Roles	Staff accounts, permissions, access control
Products	Catalog, price, tax, category and supplier mapping
Inventory	Stock quantity, reorder levels, stock movements
Customers	Customer profiles and contact info
Suppliers	Vendor master data
Orders	Sales transactions and transaction status
Order Items	Line items for each sale
Payments	Cash/card/bank transfer and tender details
Invoices	Invoice lifecycle and payment summary
Returns	Refunds and exchange return workflows
Shifts	Register sessions and cashier accountability
Activity Logs	Auditable operational changes

1. Stores

stores: id, name, code, address, city, country, timezone, status, created_at, updated_at

2. Users / Staff

users: id, store_id, full_name, email, username, employee_code, phone, role, status, last_login, created_at, updated_at

3. Roles & Permissions

roles, permissions, role_permissions: support access control for dashboard, pos, products, reports, users, settings

4. Registers

registers: id, store_id, name, status, cashier_user_id, opened_at, closed_at

5. Categories

categories: id, store_id, name, description, status, created_at, updated_at

6. Suppliers

suppliers: id, store_id, code, name, contact_person, phone, email, address, city, country, supplier_type, status, created_at, updated_at

7. Customers

customers: id, store_id, code, name, customer_type, phone, email, address, city, country, is_walk_in, status, order_count, created_at, updated_at

8. Products

products: id, store_id, category_id, supplier_id, sku, barcode, name, description, purchase_price, selling_price, tax_rate, reorder_level, status, is_active, created_at, updated_at

9. Inventory

inventory: id, product_id, current_stock, reserved_stock, reorder_level, last_updated, stock_status;
stock_movements: product_id, movement_type, quantity, reference_type, reference_id, reason, created_by, created_at

10. Orders

orders: id, store_id, register_id, customer_id, cashier_user_id, order_number, status, subtotal, discount_amount, tax_amount, total_amount, payment_method, payment_status, created_at, completed_at

11. Order Items

order_items: id, order_id, product_id, product_name, sku, quantity, unit_price, discount_amount, tax_rate, line_total, created_at

12. Payments

payments: id, order_id, payment_method, payment_status, amount, tendered_amount, change_amount, reference, processed_by, processed_at

13. Invoices

invoices: id, store_id, order_id, customer_id, invoice_number, invoice_date, due_date, subtotal, discount_amount, tax_amount, total_amount, payment_status, status, created_at

14. Returns

returns: id, store_id, order_id, customer_id, return_number, reason, notes, refund_method, subtotal, tax_amount, refund_amount, status, created_at; return_items: id, return_id, order_item_id, product_id, quantity, unit_price, refund_amount

15. Shifts

shifts: id, store_id, register_id, opened_by, opened_at, closed_by, closed_at, opening_cash, closing_cash, status

16. Activity Logs

activity_logs: id, store_id, user_id, entity_type, entity_id, action, details, created_at

Key relationships

- ? One store has many users, registers, products, customers, suppliers, orders, and invoices.
- ? One category has many products.
- ? One supplier has many products.
- ? One product has one inventory row and many stock movement records.
- ? One order has many order_items and many payments.
- ? One invoice belongs to one order.
- ? One customer may have many orders and many returns.
- ? One user can open and close shifts and perform operational actions that are logged in activity_logs.

Important design rules

- ? Use NUMERIC/DECIMAL for all money fields rather than floating-point types.
- ? Use UUIDs or bigserial as primary keys for stable references across distributed services.
- ? Apply unique constraints on sku, barcode, customer code, supplier code, invoice number, and order number.
- ? Add indexes on product lookup, orders by customer and date, inventory by product, and returns by status.
- ? Use soft delete or status flags for operational records so historical transactions are not lost.
- ? Store created_at and updated_at on all master and transactional tables for auditability.

Minimum viable schema for the current mock

If the team wants to start with the smallest useful version that matches the current mock UI, build these tables first:
stores, users, roles, permissions, categories, suppliers, customers, products, inventory, orders, order_items,
payments, invoices, returns, and return_items.

Conclusion

This front end already describes a realistic retail POS system. The database should be a normalized relational schema centered around products, inventory, orders, payments, and returns, with supporting tables for customers, suppliers, users, stores, and register activity. This architecture supports the current UI and scales for later features like low-stock alerts, shift reports, analytics, and multi-store operations.